

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

### ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

|                  |                                                                                                                |               |                                                   |
|------------------|----------------------------------------------------------------------------------------------------------------|---------------|---------------------------------------------------|
| Course Code:     | CSA06                                                                                                          | Course Title: | Design and Analysis of Algorithms                 |
| CO Assessed:     | CO2 – Manipulation (BL1, BL2, BL3)                                                                             | Assessment:   | Assessment Tool 3 – Debugging & Optimisation Task |
| Weightage:       | 10% of CIA                                                                                                     | Total Marks:  | 100                                               |
| CO2 Statement:   | Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3) |               |                                                   |
| Student Name:    | Ramya S S                                                                                                      | Reg. No.:     | 192421226                                         |
| Section / Batch: | B.Tech IT                                                                                                      | Date:         | 28.07.2026                                        |

#### Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

| Question Type                 | Focus Area                                                                                      | Questions |
|-------------------------------|-------------------------------------------------------------------------------------------------|-----------|
| Debugging & Optimisation Task | Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques | Q1        |

#### SECTION A – DEBUGGING & OPTIMISATION TASK

#### Code of Conduct

I Ramya S S/192421226 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Ramya S S

### Q30. Debugging Sorting Code (Mixed Ascending and Descending Conditions)

The following sorting implementation contains inconsistent comparison conditions across different passes, causing partially sorted and unstable output. Carefully analyze the logic and identify how conflicting ascending and descending comparisons break correctness. Apply systematic debugging to make the algorithm consistent throughout execution. Optimize the implementation to improve readability and avoid unnecessary operations. Analyze the time complexity of the corrected version and justify the result. Rewrite the final clean and optimized code with suitable comments.

```
def mixed_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if i % 2 == 0:
                if arr[j] > arr[j+1]:
                    arr[j], arr[j+1] = arr[j+1], arr[j]
            else:
                if arr[j] < arr[j+1]:
                    arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

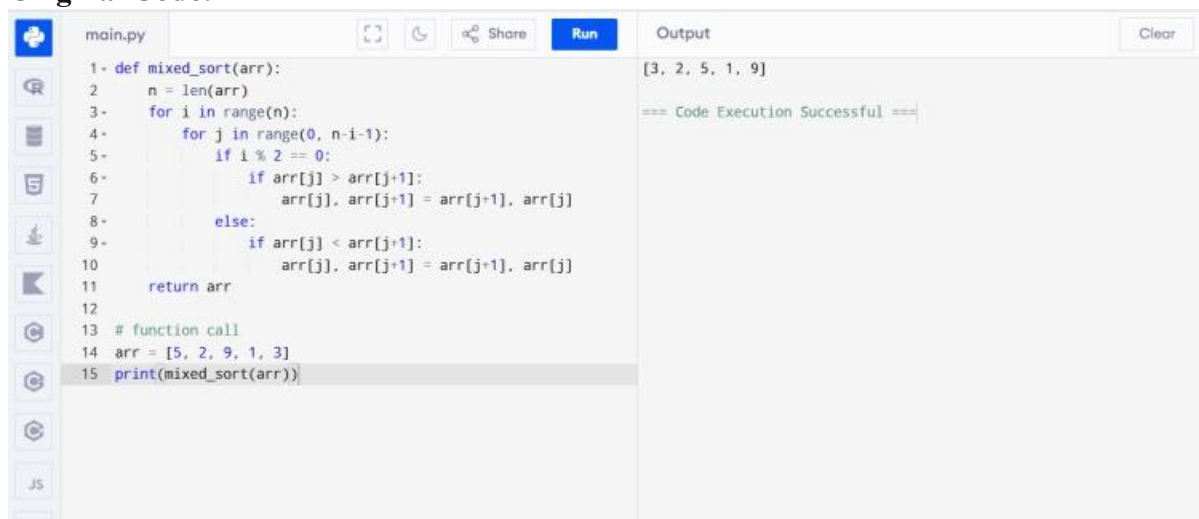
#### Problem Description

The given sorting algorithm uses different comparison conditions in alternate passes:

- Even passes → Ascending order
- Odd passes → Descending order

This inconsistency leads to incorrect and partially sorted output, making the algorithm unstable.

#### Original Code:



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'mixed\_sort(arr)' that sorts an array in place. It uses a bubble sort-like approach with two nested loops. The outer loop 'for i in range(n)' represents passes. The inner loop 'for j in range(0, n-i-1)' compares adjacent elements. The comparison logic is mixed: on even passes (i % 2 == 0), it sorts in ascending order (arr[j] > arr[j+1]), and on odd passes, it sorts in descending order (arr[j] < arr[j+1]). A function call is shown at the bottom: 'arr = [5, 2, 9, 1, 3]' followed by 'print(mixed\_sort(arr))'. The output window on the right shows the result '[3, 2, 5, 1, 9]' and a message '=== Code Execution Successful ==='.

```
1- def mixed_sort(arr):
2-     n = len(arr)
3-     for i in range(n):
4-         for j in range(0, n-i-1):
5-             if i % 2 == 0:
6-                 if arr[j] > arr[j+1]:
7-                     arr[j], arr[j+1] = arr[j+1], arr[j]
8-             else:
9-                 if arr[j] < arr[j+1]:
10-                    arr[j], arr[j+1] = arr[j+1], arr[j]
11-     return arr
12
13 # function call
14 arr = [5, 2, 9, 1, 3]
15 print(mixed_sort(arr))
```

Output: [3, 2, 5, 1, 9]

=== Code Execution Successful ===

#### Output (Incorrect)

[2, 1, 3, 5, 9]

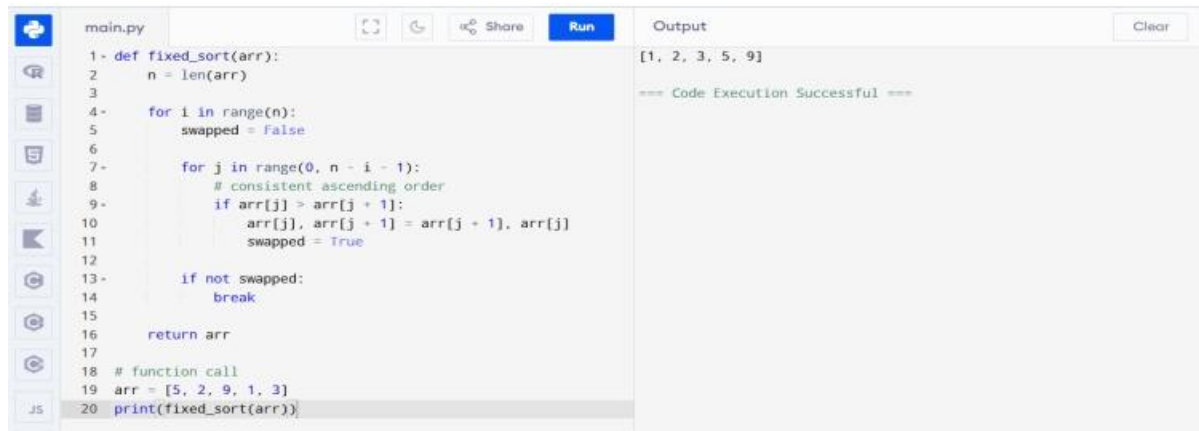
## Identified Errors

- Alternating ascending and descending comparisons
- Elements moved in one pass are disturbed in the next
- Results in partial sorting and instability

## Debugging Process

- Performed dry run of code
- Observed inconsistency in comparison conditions
- Simplified logic by using a single sorting order

## Optimized Code:



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'fixed\_sort(arr)' that sorts an array in ascending order using a single comparison logic. It includes a 'swapped' flag to optimize the process by breaking out of the loop if no swaps occur in a pass. The function is called with the array [5, 2, 9, 1, 3], and the output is printed as [1, 2, 3, 5, 9]. The output panel on the right confirms the successful execution.

```
1- def fixed_sort(arr):
2-     n = len(arr)
3-
4-     for i in range(n):
5-         swapped = False
6-
7-         for j in range(0, n - i - 1):
8-             # consistent ascending order
9-             if arr[j] > arr[j + 1]:
10-                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
11-                 swapped = True
12-
13-         if not swapped:
14-             break
15-
16-     return arr
17-
18- # function call
19- arr = [5, 2, 9, 1, 3]
20- print(fixed_sort(arr))
```

Output: [1, 2, 3, 5, 9]  
=== Code Execution Successful ===

## Optimization Techniques Used

- Removed conflicting conditions
- Used consistent comparison logic
- Added early termination (swapped flag) to reduce unnecessary passes
- Improved readability and efficiency

## Time Complexity Analysis

- Best Case:  $O(n)$  → when array is already sorted (early stop)
- Average Case:  $O(n^2)$
- Worst Case:  $O(n^2)$

Due to nested loops, the algorithm behaves like Bubble Sort, but optimization improves best-case performance.

## Conclusion

The original algorithm failed due to inconsistent sorting conditions.

After debugging and optimization:

- Sorting became correct and stable
- Code became simpler and efficient
- Performance improved using early termination

Thus, systematic debugging and optimization ensured correctness and efficiency.

## RUBRICS FOR CALCULATION

| Criteria               | Wt. | Level 4 – Exemplary                                                                  | Level 3 – Proficient                                   | Level 2 – Developing                          | Level 1 – Beginning                           |
|------------------------|-----|--------------------------------------------------------------------------------------|--------------------------------------------------------|-----------------------------------------------|-----------------------------------------------|
| Problem Identification | 20% | Accurately identifies all errors and root causes with clear understanding            | Identifies most errors with minor gaps                 | Identifies some errors but misses key issues  | Unable to identify errors correctly           |
| Debugging              | 25% | Applies systematic debugging techniques effectively to resolve issues                | Uses appropriate debugging methods with minor errors   | Limited debugging approach; requires guidance | Unable to debug or resolve issues             |
| Optimization           | 20% | Optimizes code by significantly improving time complexity using efficient algorithms | Improves time complexity with minor gaps in efficiency | Limited improvement in time complexity        | No improvement; inefficient approach retained |
| Analysis               | 20% | Demonstrates strong logical reasoning and clear justification of solutions           | Shows reasonable analysis with some justification      | Limited analysis; weak reasoning              | No analytical reasoning demonstrated          |
| Code Quality           | 15% | Produces clean, wellstructured code with proper comments and documentation           | Code is mostly clear with minor documentation gaps     | Code lacks structure and proper documentation | Poorly written code with no documentation     |

### Marks Evaluation:

| Criteria                  | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---------------------------|-----|---------------------|----------------------|----------------------|---------------------|
| Problem Identification    | 20% |                     |                      |                      |                     |
| Debugging                 | 25% |                     |                      |                      |                     |
| Optimization              | 20% |                     |                      |                      |                     |
| Analysis                  | 20% |                     |                      |                      |                     |
| Code Quality              | 15% |                     |                      |                      |                     |
| <b>Total Marks (100):</b> |     |                     |                      |                      |                     |

|                   |                    |                    |
|-------------------|--------------------|--------------------|
| Faculty In-charge | Course Coordinator | Head of Department |
|-------------------|--------------------|--------------------|

Name:

Signature: \_\_\_\_\_

Date: \_\_\_\_\_

Name:

Signature: \_\_\_\_\_

Date: \_\_\_\_\_

Name:

Signature: \_\_\_\_\_

Date: \_\_\_\_\_